

- Steigende Bedeutung der Mikroklimaforschung
  - ▶ Große Datensammlungen
- Steigende Bedeutung der Informatik
  - KI-Hype
    - ▶ Soziale / Ökologische Kosten

- Steigende Bedeutung der Mikroklimaforschung
  - Rückzugsort für bedrohte Arten
  - Resilienz in Landwirtschaft
- Herausforderungen:
  - Klimaforschung = Statistik
    - > Klimadaten (selbst für Mikroklimaforschung) umfassen riesige Gebiete (ganze Städte, kleinere Wälder, ganz Brandenburg)
    - > Also: Computer
  - Außerdem **große Zeiträume**

Siehe nächste N



- Steigende Bedeutung der Mikroklimaforschung
  - Große Datensammlungen
- Steigende Bedeutung der Informatik
  - KI-Hype
    - Soziale / Ökologische Kosten

- Steigende Bedeutung der Informatik
  - etwas selbsterklärend
  - Digitalisierung
  - Bürokratiebewältigung
- KI-Hype
  - Medienwirksam
  - Hohe \*soziale\* + \*ökologische\* Kosten

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
- Verlässliche, hochwertige Software erforderlich

⇒ Softwaredesign von zentraler Bedeutung

- Aufgabe zur Entwicklung einer Optimierungssoftware für Klima-Maßnahmen
  - Klima-Maßnahmen
    - > Werden auf Flächen angewandt
    - > Flächen müssen gekauft werden
    - > Bsp: Bestandsbegrünung, Anlegen von Baumgruppen
    - > Werden optimiert
- Verlässliche, hochwertige Software erforderlich
  - Unfassbare Geldbeträge (hunderte Millionen €) laufen über software
    - > Kleine Fehler verursachen Schäden in Millionenhöhe
  - Verwendung in der Regierung
  - Auch von Menschen ohne Computeraffinität

# IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

## Einführung Softwaredesign

### Softwaredesign – Einführung

|                              |                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------|
| Software-Design              | Das Planen von Software um bestimmte Ziele zu erfüllen                                               |
| Design Patterns <sup>1</sup> | Standard Lösungen für Probleme bei der Softwareentwicklung<br>Werden entdeckt, <u>nicht</u> erfunden |
| Paradigmen                   | Fundamentale Programmierstile                                                                        |

---

<sup>1</sup> „Entwurfsmuster“

Software-Design      Das Planen von Software um bestimmte Ziele zu erfüllen

Design Patterns<sup>2</sup>      Standard Lösungen für Probleme bei der Softwareentwicklung  
Patterns werden entdeckt, nicht erfunden

Paradigmen              Fundamentale Programmierstile

- Testbarkeit
  - ▶ Automatisierbarkeit
- Wartbarkeit
- Erweiterbarkeit
- Kompatibilität

⇒ Standards setzen / Entscheidungen treffen

## Welche Ziele soll die Software nun erfüllen?

- Testbarkeit
    - Automatisierbarkeit
    - Wie gesagt Fehler teuer
  - Wartbarkeit und Erweiterbarkeit
    - Dinge im Code ändern sollte einfach sein
    - Andere Menschen sollen Projekt weiterentwickeln können
  - Kompatibilität
    - Windows/MacOs/Linux/Web
- ⇒ wie machen wir das? **Standards setzen** / Entscheidungen treffen

**Überleitung xkcd**



<https://xkcd.com/927/>

- Also: Keine Standards selber setzen wenn es bereits Standards gibt!
- Diese „Standards“ finden wir als Design-Patterns
- Um uns eine **Meinung über Designpattern** (Standardlösungen) zu bilden müssen wir uns **Paradigmen** (Programmierstile ansehen)

# IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

## └ Einführung Coding-Paradigmen

### └ Paradigmen – Einführung

Unterteilung:

- Imperative-Programmierung
  - Objekt-Orientierte-Programmierung (OOP)
- Deklarative-Programmierung

Unterscheidung: **State-Frage!**

Unterteilung:

- **Imperative**-Programmierung (Kennt ihr schon)
  - **Objekt-Orientierte**-Programmierung (OOP) (Kennt ihr auch schon)
- **Deklarative**-Programmierung

Unterscheidung: **State-Frage!**

# IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

## └ Einführung Coding-Paradigmen

### └ Paradigmen – State-Frage

Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

State Frage: Wie wird **state** in einer Anwendung gehandhabt?

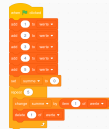
Was ist State?

- Anwendungszustand
- Die Gesamtheit des Zustands in dem sich ein Programm befindet

State Frage: Wie wird **state** in einer Anwendung gehandhabt?



```
werte = [1,2,3,4,5]  
summe = 0  
for wert in werte:  
    summe = summe + wert
```



- Beispiel
- Auch gutes Beispiel für die Mächtigkeit von Python vs Scratch

## Imperative Programmierung

- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
- Beschreibt wird **wie** etwas durchgeführt wird

- Kennt ihr schon
- Reguläre Programmierung, alles was für euch halbwegs intuitiv ist
- Keine Klassen
- Untertypen
  - z.b. methodische Programmierung (funktionen)
- Anwendungszustand muss bei jeder Anweisung berücksichtigt werden
  - state ist extern zum code
- Beschreibt wird **wie** etwas durchgeführt wird
  - Wir geben den Computer eine Anleitung
  - klingt so als gäbe es keine alternative, gibt es aber

```
class Person
    def __init__(self, name):
        self.name = name
    def greet(self):
        print(self.name + " says hello!")

ada = Person("Ada Lovelace")
ada.greet()

alan = Person("Alan Turing")
alan.greet()
```

- Bsp

## Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
  - Kein globaler<sup>1</sup> State; State ist lokal<sup>2</sup>
- Beschreibt wird **wie** etwas durchgeführt wird

<sup>1</sup> global := von überall sichtbar/veränderbar  
<sup>2</sup> lokal := nur lokal sichtbar/veränderbar

## Objekt-Orientierte Programmierung

- Anwendungszustand wird in Klassen gekapselt (Datenkapselung)
  - Kein globaler State!
  - d.h. State sollte nicht unnötig außerhalb derselben Klasse verfügbar sein.
    - > Globaler state: Überall sichtbar/veränderbar
    - > lokaler state: Nur lokal (z.B. in der selben Klasse) sichtbar
- Beschreibt wird **wie** etwas durchgeführt wird

- (Veränderbarer) Anwendungszustand existiert nicht.

► Setzen wir einmal eine variable dürfen wir sie nicht verändern

► State wird mit Hilfe von Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```

a = 1          def main():          import time
def main():    a = 1                def timestamp_add_one():
    print(a*2)  b = 2                def timestamp_add_one():
                                    return ta + 1
                                print(a)

```

## Paradigmen Deklarative Programmierung

### Deklarative Programmierung

#### Zielsetzung:

- (Veränderlicher) Anwendungszustand existiert nicht.
  - Setzen wir einmal eine variable dürfen wir sie nicht verändern
  - State wird in Funktionen „gekapselt“, state darf nicht außerhalb von Funktionen erscheinen

```

a = 100
def main():
    a = 1
    b = 2
    print(a+b)

```

```

def main():
    a = 1
    b = 2
    c = a + b
    print(c)

```

```

def add(a,b):
    return a + b

```

```

def main():
    a = 1
    b = 2
    print(add(a,b))

```

# IAWAKEE – Lach und Sachgeschichten aus Mikroklimaforschung und Informatik

## └ Einführung Coding-Paradigmen

### └ Deklarative Programmierung

#### └ Paradigmen – Deklarative Programmierung

```
def main(werte):  
    summe = 0  
    for wert in wert:  
        summe = summe + wert  
    return summe
```

```
def main(werte):  
    return sum(werte)
```

- Links: Imperative Programmierung
    - Gehe werte in der Liste Durch
      - > Rechne den aktuellen Listenwert auf die Summe hinzu
    - Gebe Summe zurück
  - Rechts: Deklarative Programmierung
    - Das Ergebnis ist die Summe aller Werte.
- Beschreiben des Ergebnisses statt

```
let wert = [0.2912, 0.421, ...];  
let res = wert.iter()  
    .map(|wert| wert * 100)  
    .map(|wert| wert.round())  
    .filter(|i| i > 50)  
    .fold(0, |avg, i| avg + i / wert.len());  
  
SELECT AVG(gerundeter_wert) AS durchschnitt_gefiltert  
FROM (  
    SELECT ROUND(wert * 100.0) AS gerundeter_wert  
    FROM werte_tbl  
) t WHERE gerundeter_wert > 50;
```

1. Das ergebnis ist die Repräsentation aller Werte als Prozent-, statt Kommazahlen
2. Diese Prozentzahlen sind gerundet
3. Nur Prozentzahlen  $> 50$  werden ausgewählt
4. Von den ausgewählten Zahlen wird der Durchschnitt errechnet.
5. SQL ebenfalls deklarativ!

```
werte = [0.2012, 0.421, ...]

def to_percent(wert):
    return wert * 100
def rounded(wert):
    return round(wert)
def is_gt_50(wert):
    return wert > 50
def build_avg(length):
    def avg(acc, wert):
        return acc + wert / length
    return avg

res = Werte.map(to_percent)
               .map(rounded)
               .filter(is_gt_50)
               .fold(0, build_avg(len(werte)))

werte = [0.2012, 0.421, ...]

avg = 0
for wert in werte:
    a = wert * 100
    b = round(a)
    if b > 50:
        c = b
    avg = avg + c / len(werte)
```

- etwas Gefährlich
- IdR nicht wirklich optimiert, aber möglich!
- Rust z.B. kann eigentlich rekursive Prozesse → iterativ umsetzen
- Rekursive Prozesse lassen sich problemlos iterativ darstellen



### └ Pattern in der Objekt-Orientierten Programmierung

#### Factory Method Pattern

```
public interface Postage { String type(); }

public class RegularPostage implements Postage {
    public String type() { return "Regular"; }
}

public class ExpressPostage implements Postage {
    public String type() { return "Express"; }
}

public final class PostageFactory {
    private PostageFactory() {}
    public static Postage by_price(double price_for_product) {
        if (price_for_product > 30) {
            return new ExpressPostage();
        } else {
            return new RegularPostage();
        }
    }
}
```

- Wenn verschiedene Klassen ein Interface implementieren können wir zur Laufzeit entscheiden ein Objekt welcher konkreten Klasse wir zurückgeben
- Methode by\_price von Klasse PostageFactory gibt interface Postage zurück.
  - Methode heißt Factory method / „fabrik methode“

### └ Pattern in der Objekt-Orientierten Programmierung

#### Builder Pattern

```

class House:
    def __init__(self, walls, roof, door):
        self.walls = walls
        self.roof = roof
        self.door = door

class HouseBuilder:
    def __init__(self):
        self.walls = "concrete" # House have concrete walls by default
        self.roof = "tile" # House have tile roof by default
        self.door = "glass" # House have glass by default
    def walls(self, style):
        self.walls = style
        return self
    def roof(self, style):
        self.roof = style
        return self
    def door(self, style):
        self.door = style
        return self
    def build(self):
        return House(self.walls, self.roof, self.door)

```

- Hier Übertragen wir die Verantwortung die Klasse House zu konstruieren an die Klasse HouseBuilder
- Dadurch müssen wir den Konstruktor von House nicht mehr selber aufrufen und können dadurch Standard Werte festlegen.
- Unnötig in python in diesem kontext

- Objektorientierte Programmierung
  - ▶ State sollte in Klassen gekapselt sein
  - ▶ Globalen State vermeiden
- Deklarative Programmierung
  - ▶ State allgemein vermeiden
  - ▶ State-Änderungen vermeiden
  - ▶ Deklarative-Primitives nutzen
  - ▶ Immutability > Datenkapselung
- Design Pattern
  - ▶ Bieten Struktur, Standardisierung und Namenskonvention

- Am Ende kommt es auf die State Frage an
- Objektorientierte Programmierung
  - State sollte in Klassen gekapselt sein
  - Globaler State ist zu vermeiden
- Deklarative Programmierung
  - State-Änderungen / Änderbarer State ist zu vermeiden
  - Deklarative-Primitives nutzen
    - > Funktionen wie reduce, forEach, etc. werden von Sprache idR. bereits bereitgestellt
  - Zu starke Gewichtung dieser Ideale in den meisten Kontexten schwierig, etwas Deklarative Programmierung macht Code häufig besser
  - Immutability > Datenkapselung
- Design Pattern

